COMPUTER SCIENCE PROJECT
ACADEMIC YEAR-2024-25



TOPIC: ALUMNI MANAGEMENT SYSTEM

SUBMITTED TO
MS. DIVYA SAINI

SUBMITTED BY
SHRI SAMYUKTHA
SHIVARAMAN

ACKNOWLEDGEMENT

I would like to express my special thanks and gratitude to our computer teacher Ms. Divya Saini who gave us this opportunity to do this wonderful project and gave exemplary guidance, monitoring and constant encouragement throughout the course of this project.

We also take this opportunity to express a deep sense of gratitude to Co-Ordinator Ms. Reshma Bhattacharjee for their cordial support, valuable information and guidance, which helped in completing this project through various stages.

INDEX

S.NO	TOPIC	PGNO.
1	Brief overview of the project	2
2	Objective and scope	3
3	Flow	4
4	Hardware and software requirements	8
5	Python libraries	8
6	Source code	10
7	Output screen	28
8	Conclusion	43
9	Bibliography	44

Brief overview of the project

The Alumni Management Program is a comprehensive solution designed to facilitate and enhance engagement between educational institutions and their alumni. Recognizing the importance of alumni in providing a network for mentorship, career opportunities, and institutional growth, this program is structured to streamline administrative processes while fostering meaningful connections with former students. It serves as a centralised platform where alumni can register, update their details, view events, and participate in institution-led activities. For administrators, it provides a secure and efficient way to manage alumni data, organise events, and communicate effectively.

The system is built with user-friendly, menu-driven interfaces tailored to the needs of both alumni and administrators. Through its dual interface, the program ensures that alumni can seamlessly interact with event information and registration processes, while administrators have tools for maintaining accurate records, generating engagement opportunities, and upholding data security. Designed to be robust yet intuitive, the program employs secure login mechanisms, data validation, and MySQL-based data handling to meet the high standards of data integrity and security required in educational institutions. By integrating functionalities like email notifications, attendance management for events, and flexible search options, this Alumni Management Program is positioned as a modern solution to strengthen alumni relations

Objective and Scope

The primary aim of this Alumni Management Program is to facilitate efficient management of alumni data and improve institutional engagement with graduates. The system allows for easy alumni registration, event creation, and seamless retrieval of alumni details, making it an essential tool for alumni relations.

To enhance the functionality, scalability, and usability of this Alumni Management System:

- Maintain logs of all admin and user actions for accountability.
- Migrate the database and application to the cloud for better scalability and reliability.
- Analyse event success based on metrics like attendance, feedback, and cost efficiency.
- Track alumni activities (e.g., event participation, profile updates) and generate engagement reports.
- Convert the program into a web application for easier access and usability.
- Send automated notifications (via email or SMS) for event reminders, updates, or cancellations.
- Collect feedback after events to improve future engagements.
- Increase security by integrating OTP or email-based verification for admin and alumni logins

Flow

1. User Authentication:

- **Admin Login**: Secure login for administrators using username and password validated against the database.
- **Attempt Limit**: Limits login attempts to three, enhancing security.

2. Student Management:

- **Registration**:
  - Registers students with details like name, class, DOB, contact info, email, passing year, stream, location, and employment details.
  - Validates age (DOB) and email format during registration.
- **Search**:
  - Filters students based on various criteria like name, passing year, stream, and employment status.
  - Displays search results in a structured tabular format.
- **Update**:
  - Allows modification of specific student details such as class, contact number, email, employment status, domain, or company.

3. Event Management:

- **Create Events**:
  - Administrators can create new events by specifying details such as event name, date, type (e.g., workshop, seminar), venue, seats, and status.
  - Validates the event date to ensure it's scheduled at least two weeks from the current date.

- **Update Events**:
    - Modifies event details, including name, description, date, and other properties, while validating inputs.
- **Search Events**:
    - Enables filtering of events by their status (active, completed, postponed, cancelled).

4. Communication:

- **Email Functionality**:
    - Sends emails to alumni or students with updates or notifications.
    - Validates email addresses before sending.
    - Uses SMTP for sending emails with a pre-configured email account.

5. Alumni Registration:

- **Validation with Student Records:**
    - Cross-verifies alumni details with existing student records in the database.
    - Ensures only valid alumni can register.
- **Duplicate Registration Prevention:**
    - Checks for existing registrations to avoid redundant entries in the alumni database.
- **Database Updates:**
    - Transfers data from student records to the alumni database while allowing updates like contact information and employment status.

6. Event Management:

- **View Events:**
  - Provides categorised views of events:
    - Upcoming (Active).
    - Completed.
    - Postponed.
    - Cancelled
- **Event Registration:**
  - Displays active events with available seats in a user-friendly tabular format.
  - Enables alumni to register for events while dynamically updating seat availability in the database.

7. Alumni Search:

- Allows alumni to search for other registered alumni using:
  - Name.
  - Email.
  - Passing Year.
- Displays search results in a well-structured table format for clarity and ease of access.

8. Integration with Admin Features:

- The program complements admin functionality by enabling seamless alumni interactions with events and records created or updated by administrators.

9. Usability and Navigation:

- **Menu Options:**
  - Easy-to-navigate options to register, view events, register for events, or exit the program.
- **Error Handling:**
  - Validates inputs like email addresses and numeric values to ensure data integrity.

10. Secure Database Interaction:

- Executes secure SQL queries to:
  - Fetch events.
  - Register alumni for events.
  - Verify and add alumni details.
- Maintains data consistency and security during interactions.

11. Exit and Main Menu Options:

- Offers flexibility to return to the main menu or exit the program, providing a smooth user experience.

12. Database Interaction:

- **MySQL Integration**:
  - Connects to a MySQL database to store and retrieve information about students, alumni, and events.
  - Executes secure queries to prevent SQL injection.

13. User-Friendly Interface:

- Displays clear options in menus for various tasks (e.g., registration, updates, event creation).

- Provides intuitive navigation between sections with error handling for invalid inputs.

Hardware and software requirements

1. Hardware-
    - PC
2. Software-
    - Python (IDLE/Visual Studio)
    - MYSQL command line client
    - MYSQL workbench
    - Python connector

Python libraries

- **PrettyTable()** – Python library used for displaying data in a tabular format, enhancing readability for search results and event listings.
- **datetime** – Python library used for managing dates and times, such as scheduling events, calculating ages, and tracking registration dates.
- **getpass** – Python library used for securely handling password input during the admin login process.
- **smtplib** – Python library used for sending emails via SMTP, facilitating communication with alumni and administrators.
- **mysql.connector** – Python library used for connecting to MySQL databases, executing queries, and performing CRUD operations on student, alumni, and event records.
- **regex (re)** – Python library used for validating email formats through regular expressions.
- **sys** – Python library (implicitly used) for program exits and handling command-line arguments.

- **utils (custom module)** – Custom utility library providing reusable functions like get_input, is_valid_email, and login for input handling, validation, and authentication

Source code

```
from utils import login
login()
```

```
def is_valid_email(email):
    import re
    email = email.strip()
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}
$'  return re.match(pattern, email) is not None
```

```
def login():
    user = input("Enter A(Admin) or B(Alumni):
")  if user.upper() == "A":
        from admin import Admin
        Admin()
    elif user.upper() == "B":
        from alumni import Alumni
        Alumni()
    else:
        print("Invalid Input!")
```

```
def get_user_input(prompt, is_int=False, is_name=False, is_date=False):
    import datetime
    value = input(prompt)
    if value.strip() == "":
        return None
    if is_int:
        try:
            return int(value)
        except ValueError:
            print("Invalid input. Please enter a valid integer.")
            return get_user_input(prompt, is_int=True)
    elif is_name:
        if all(part.isalpha() for part in value.split()):
            return value
        print("Invalid input. Please enter a valid name (letters only).")
        return get_user_input(prompt, is_name=True)
    elif is_date:
        try:
            datetime.datetime.strptime(value, '%Y-%m-%d')
            return value
        except ValueError:
            print("Invalid date format. Please use YYYY-MM-DD.")
            return get_user_input(prompt, is_date=True)
    return value
```

```
def get_input(prompt, is_int=False, is_name=False, is_date=False):
    import datetime
    while True:
        value = input(prompt)
        if is_int:
            try:
                return int(value)
            except ValueError:
                print("Invalid input. Please enter a valid integer.")
        elif is_name:
            if value.isalpha() or " " in value:
                return value
            print("Invalid input. Please enter a valid name (letters onl
y).")   elif is_date:
            try:
                datetime.datetime.strptime(value, '%Y-%m-%d')
                return value
            except ValueError:
                print("Invalid date format. Please use YYYY-MM-DD.")
        else:
            if value.strip():
                return value
            print("Invalid input. Please enter a valid value.")
```

```
def Admin():
    from utils import login
    login_successful = adminlog()
    if login_successful:
        print('-------WELCOME----------')
        print('''Please Select an Option
              \n1. Student Registration
              \n2. Update Student Data
              \n3. Create Event
              \n4. Update Event
              \n5. Send Mail
              \n6. Search Students
              \n7. Main Menu
              \n8. Exit''')
        print("\n\n\n")
        choice = int(input('Enter a Choice (1-8):
'))
        if choice == 1:
            register_stu()
        elif choice == 2:
            update_stu()
        elif choice == 3:
            create_event()
        elif choice == 4:
            update_event()
        elif choice == 5:
            send_email_message()
        elif choice == 6:
            search()
        elif choice == 7:
            login()
        elif choice == 8:
            exit()
        else:
            print('Invalid input!')
    else:
        print('Login failed. Please try again.')
```

```
def Admin2():
    from utils import login
    login_successful = True
    if login_successful:
        print('-------WELCOME----------')
        print('''Please Select an Option
              \n1. Student Registration
              \n2. Update Student Data
              \n3. Create Event
              \n4. Update Event
              \n5. Send Mail
              \n6. Search Students
              \n7. Main Menu
              \n8. Exit''')
        print("\n\n\n")
        choice = int(input('Enter a Choice (1-8):
'))
        if choice == 1:
            register_stu()
        elif choice == 2:
            update_stu()
        elif choice == 3:
            create_event()
        elif choice == 4:
            update_event()
        elif choice == 5:
            send_email_message()
        elif choice == 6:
            search()
        elif choice == 7:
            login()
        elif choice == 8:
            exit()
        else:
            print('Invalid input!')
    else:
        print('Login failed. Please try again.')
```

```
def adminlog():
    from utils import login
    import mysql
    import mysql.connector
    import getpass

    print('-------------------------LOGIN-----------------------------')
    db = None
    cursor = None
    max_attempts = 3
    attempts = 0

    while attempts < max_attempts:
        try:
            username = input("Enter your username: ")
            password = getpass.getpass("Enter your password: ")

            db = mysql.connector.connect(host="localhost", user="root", password="1234", database="alumn
i")         cursor = db.cursor()

            query = "SELECT * FROM admin_login WHERE username=%s AND password=%s"
            cursor.execute(query, (username, password))

            login = cursor.fetchone()

            if login is not None:
                print("Login successful!")
                return True
            else:
                attempts += 1
                print(f"Invalid Login! Attempts remaining: {max_attempts - attempts}")

        except mysql.connector.Error as err:
            print("Error: ",err)

        finally:
            if cursor:
                cursor.close()
            if db:
                db.close()

    print("Maximum login attempts exceeded. Exiting the program.")
    exit()
```

```
def register_stu():
    from utils import get_input
    import datetime
    from utils import is_valid_email
    import mysql.connector
    from prettytable import PrettyTable
    Name = get_input("Enter the name of the student: ", is_name=True)
    M_Name = get_input("Enter the M_Name: ", is_name=True)
    F_Name = get_input("Enter the F_Name: ", is_name=True)
    Class = get_input("Enter the class: ")
    while True:
        DoB = get_input("Enter the DOB (YYYY-MM-DD): ", is_date=True)
        dob_dt = datetime.datetime.strptime(DoB,'%Y-%m-%d')
        today = datetime.datetime.now()
        min_valid_date = today.replace(year=today.year - 14)
        if dob_dt > min_valid_date:
            print("Student age is less than 14. Cannot register as an alumni.")
        else:
            break
    Contact_Number = get_input("Enter the contact number: ")

    while True:
        email_input = get_input("Enter the email: ")
        if is_valid_email(email_input):
            Email_ID = email_input
            break
        else:
            print("Invalid email format. Please try again.")

    Passing_Year = get_input("Enter the passing year: ", is_int=True)
    Stream = get_input("Enter the stream taken: ")
    Current_Country = get_input("Enter the Current country of the student: ")
    Current_City = get_input("Enter the current city of the student: ")
    Employment_Status = get_input("Enter Employment status (e.g., Employed, Unemployed, etc.): ")
    E_Domain = get_input("Enter Domain of work (e.g., HR, Engineer, etc. OR N/A): ")
    Company = get_input("Enter the company OR N/A: ")

    db = mysql.connector.connect(host="localhost", user="root", password="1234", database="alumni")
    cursor = db.cursor()

    command = '''INSERT INTO students (Name, M_Name, F_Name, Class, DOB, Contact_Number, Email_ID, Passing_Year,
            Stream, Current_Country, Current_City, Employment_Status, E_Domain, Company) VALUES (%s, %s, %s, %s, %s, %s,
            %s, %s, %s, %s, %s, %s, %s, %s)'''

    cursor.execute(command, (Name, M_Name, F_Name, Class, DoB, Contact_Number, Email_ID, Passing_Year,
                             Stream, Current_Country, Current_City, Employment_Status, E_Domain, Company))

    db.commit()

    cursor.execute("SELECT * FROM students WHERE Email_ID = %s", (Email_ID,))
    result = cursor.fetchone()

    table = PrettyTable()
    table.field_names = ["ID", "Name", "M_Name", "F_Name", "Class", "DOB", "Contact_Number", "Email_ID", "Passing_Yea
r",                      "Stream", "Current_Country", "Current_City", "Employment_Status", "E_Domain", "Company"]
    table.add_row(result)

    print("Student registered successfully!")
    print(table)

    cursor.close()
    db.close()
    Admin2()
```

```
def update_stu():
    from utils import get_user_input
    from utils import is_valid_email
    import mysql.connector
    from prettytable import PrettyTable

    db = mysql.connector.connect(host="localhost", user="root", password="1234", database="alumni")
    cursor = db.cursor()

    cursor.execute("SELECT * FROM students")
    students = cursor.fetchall()

    if not students:
        print("No students found.")
        cursor.close()
        db.close()
        return

    table = PrettyTable()
    table.field_names = [desc[0] for desc in cursor.description]
    for student in students:
        table.add_row(student)

    print("Students List:")
    print(table)

    choice = get_user_input("Select a student by ID to update: ", is_int=True)
    student_id = students[choice - 1][0]

    print("----------------Select option to update-----------------")
    print(
        "1: Class\n"
        "2: Contact Number\n"
        "3: Email_ID\n"
        "4: Stream\n"
        "5: Current Country\n"
        "6: Current City\n"
        "7: Employment Status\n"
        "8: Employment Domain\n"
        "9: Company\n"
   )

    option = get_user_input("Please enter your Option (1-9): ", is_int=True)

    if option == 1:
        new_value = get_user_input("Enter the class: ")
        command = "UPDATE students SET Class = %s WHERE student_id = %s"
    elif option == 2:
        new_value = get_user_input("Enter the new contact number: ", is_int=True)
        command = "UPDATE students SET Contact_Number = %s WHERE student_id = %s"
    elif option == 3:
        new_value = input("Enter the new email ID: ")
        if not is_valid_email(new_value):
            print("Invalid email format. Please try again.")
            cursor.close()
            db.close()
            return
        command = "UPDATE students SET Email_ID = %s WHERE student_id = %s"
    elif option == 4:
        new_value = get_user_input("Enter the stream: ")
        command = "UPDATE students SET Stream = %s WHERE student_id = %s"
    elif option == 5:
        new_value = get_user_input("Enter the new current country: ")
        command = "UPDATE students SET Current_Country = %s WHERE student_id = %s"
    elif option == 6:
        new_value = get_user_input("Enter the new current city: ")
        command = "UPDATE students SET Current_City = %s WHERE student_id = %s"
    elif option == 7:
        new_value = get_user_input("Enter Employment status (e.g., Employed, Unemployed, etc. OR N/A):
")      command = "UPDATE students SET Employment_Status = %s WHERE student_id = %s"
    elif option == 8:
        new_value = get_user_input("Enter Domain of work (e.g., HR, Engineer, etc. OR N/A): ")
        command = "UPDATE students SET E_Domain = %s WHERE student_id = %s"
    elif option == 9:
        new_value = get_user_input("Enter the Company Name or N/A ")
        command = "UPDATE students SET Company = %s WHERE student_id = %s"
    else:
        print("Invalid option.")
        cursor.close()
        db.close()
        return

    cursor.execute(command, (new_value, student_id))
    db.commit()

    cursor.execute("SELECT * FROM students WHERE student_id = %s", (student_id,))
    updated_student = cursor.fetchone()
    table = PrettyTable()
    table.field_names = [desc[0] for desc in cursor.description]
    table.add_row(updated_student)
    print(table)

    cursor.close()
    db.close()
    print("Student data updated successfully.")
    Admin2()
```

```
def create_event():
    from utils import get_input
    import mysql.connector
    from datetime import datetime, timedelta
    from prettytable import PrettyTable

    db = mysql.connector.connect(host='localhost', user='root', password='1234', database='alumni')
    cursor = db.cursor()

    event_name = input("Enter the Event Name: ")

    while True:
        event_date_input = get_input('Enter Date of Event (YYYY-MM-DD): ', is_date=True)
        try:
            event_date = datetime.strptime(event_date_input, '%Y-%m-%d').date()
            if event_date < datetime.now().date() + timedelta(weeks=2):
                print("Event date must be at least 2 weeks from today's date. Please try again.")
            else:
                break
        except ValueError:
            print("Invalid date format. Please use YYYY-MM-DD.")

    type_event = input("Enter the type of event (e.g., Seminar, Workshop, Conference): ")
    venue = input("Enter the venue: ")

    while True:
        try:
            total_seats = int(input("Enter the total number of seats: "))
            available_seats = int(input("Enter the seats available: "))
            if available_seats > total_seats:
                print("Available seats cannot exceed the total number of seats. Please enter again.")
            else:
                break
        except ValueError:
            print("Invalid input. Please enter a valid integer for the number of seats.")

    status = input("Enter the status (Active, Cancelled, Postponed, Completed): ")

    command = '''INSERT INTO event (Event_Name, Event_Date, Type, Venue, Total_Seats, Available_Seats, Status)
                 VALUES (%s, %s, %s, %s, %s, %s, %s)'''

    try:
        cursor.execute(command, (event_name, event_date, type_event, venue, total_seats, available_seats, statu
s))     db.commit()

        cursor.execute("SELECT * FROM event WHERE Event_Name = %s AND Event_Date = %s", (event_name, event_date))
        created_event = cursor.fetchone()

        table = PrettyTable()
        table.field_names = [desc[0] for desc in cursor.description]
        table.add_row(created_event)

        print("Event created successfully.")
        print(table)
    except mysql.connector.Error as err:
        print("Error: ",err)
    finally:
        cursor.close()
        db.close()
        Admin2()
```

```
def update_event():
    import mysql.connector
    from datetime import datetime
    from prettytable import PrettyTable

    db = mysql.connector.connect(host='localhost', user='root', password='1234', database='alumni')
    cursor = db.cursor()

    cursor.execute("SELECT * FROM event")
    all_events = cursor.fetchall()

    if all_events:
        table = PrettyTable()
        table.field_names = [desc[0] for desc in cursor.description]
        for event in all_events:
            table.add_row(event)
        print("Existing Events:")
        print(table)
    else:
        print("No events found.")
        cursor.close()
        db.close()
        return

    event_id = input("\nEnter the Event ID to update: ")
    cursor.execute("SELECT * FROM event WHERE Event_ID = %s", (event_id,))
    event = cursor.fetchone()

    if not event:
        print("Event not found.")
        cursor.close()
        db.close()
        return

    valid_statuses = ["Active", "Cancelled", "Postponed", "Completed"]

    status = input("Enter the new status (Active, Cancelled, Postponed, Completed) or press Enter to keep current: ")
    if status.strip() == "":
        status = event[7]
    elif status not in valid_statuses:
        print("Invalid status. Please enter one of the following: Active, Cancelled, Postponed, Completed.")
        cursor.close()
        db.close()
        return

    while True:
        event_date_input = input("Enter new Date of Event (YYYY-MM-DD) or press Enter to keep current: ")
        if event_date_input.strip() == "":
            event_date = event[2]
            break
        try:
            event_date = datetime.strptime(event_date_input, '%Y-%m-%d').date()
            break
        except ValueError:
            print("Invalid date format. Please use YYYY-MM-DD.")

    venue = input("Enter the new venue or press Enter to keep current: ")
    if venue.strip() == "":
        venue = event[4]

    command = '''UPDATE event
                 SET Status = %s, Event_Date = %s, Venue = %s
                 WHERE Event_ID = %s'''

    try:
        cursor.execute(command, (status, event_date, venue, event_id))
        db.commit()
        print("Event updated successfully.")
    except mysql.connector.Error as err:
        print("Error:", err)
        cursor.close()
        db.close()
        return

    cursor.execute("SELECT * FROM event WHERE Event_ID = %s", (event_id,))
    updated_event = cursor.fetchone()

    table = PrettyTable()
    table.field_names = ["Event ID", "Event Name", "Event Date", "Type", "Venue", "Total Seats", "Available Seats", "Statu
s"] table.add_row(updated_event)

    print("\nUpdated Event Details:")
    print(table)

    cursor.close()
    db.close()
    Admin2()
```

```
def search_alumni(name=None, email=None, passing_year=None):
    try:
        import mysql.connector
        from prettytable import PrettyTable

        db = mysql.connector.connect(host='localhost', user='root', password='1234', database='alumn
i')     cursor = db.cursor()
        query = "SELECT alumni_id, Name, Email_ID, Passing_Year FROM alumni_students WHERE 1=1"
        parameters = []
        if name:
            query += " AND Name = %s"
            parameters.append(name)
        if email:
            query += " AND Email_ID = %s"
            parameters.append(email)
        if passing_year:
            query += " AND Passing_Year = %s"
            parameters.append(passing_year)
        cursor.execute(query, tuple(parameters))
        results = cursor.fetchall()
        if results:
            table = PrettyTable()
            table.field_names = ["Alumni ID", "Name", "Email ID", "Passing Year"]
            for row in results:
                table.add_row(row)
            print("\nSearch Results:")
            print(table)
            return results
        else:
            print("\nNo matching alumni found.")
            return []
    except mysql.connector.Error as e:
        print("Database error:", e)
        return None
    except Exception as e:
        print("An unexpected error occurred:", e)
        return None
    finally:
        if cursor:
            cursor.close()
        if db:
            db.close()
```

```
def send_email_message():
    import smtplib
    from prettytable import PrettyTable
    import mysql.connector

    # Connect to the database
    db = mysql.connector.connect(host="localhost", user="root", password="1234", database="alumni")
    cursor = db.cursor()

    # Fetch active events
    cursor.execute("SELECT Event_ID, Event_Name, Event_Date, Venue FROM event")
    active_events = cursor.fetchall()

    if not active_events:
        print("No events found.")
        cursor.close()
        db.close()
        Admin2()

    # Display events in a table
    event_table = PrettyTable()
    event_table.field_names = ["Event ID", "Event Name", "Event Date", "Venue"]
    for event in active_events:
        event_table.add_row(event)

    print("\nActive Events:")
    print(event_table)

    send_to_all = input("Do you want to send details of these active events to the alumni? (Y/N): ").strip().upper
()  if send_to_all != "Y":
        print("Aborting email sending process.")
        cursor.close()
        db.close()
        Admin2()

    # Fetch selected alumni details
    student_data = search_alumni()

    if not student_data:
        print("No students found to send emails.")
        cursor.close()
        db.close()
        Admin2()

    recipients = []
    student_table = PrettyTable()
    student_table.field_names = ["ID", "Name", "Email"]

    for student in student_data:
        student_id = student[0]
        recipient_name = student[1]
        recipient_email = str(student[2])
        student_table.add_row([student_id, recipient_name, recipient_email])
        recipients.append((recipient_name, recipient_email))

    print("\nSelected Alumni:")
    print(student_table)

    # Ask for confirmation before sending emails
    confirm_send = input("Do you want to send emails to the selected alumni? (Y/N): ").strip().upper()
    if confirm_send != "Y":
        print("Aborting email sending process.")
        cursor.close()
        db.close()
        Admin2()

    try:
        # Set up email server
        smtp_server = "smtp.gmail.com"
        smtp_port = 587
        sender_email = "projects.euroschool@gmail.com"
        app_password = "nznx fzpw hvcp mgyd"

        server = smtplib.SMTP(smtp_server, smtp_port)
        server.starttls()
        server.login(sender_email, app_password)

        # Email content
        subject = "Exciting New Events for Alumni"
        events_info = ""
        for event in active_events:
            events_info += event[1] + " on " + str(event[2]) + " at " + event[3] + "\n"
        body_template = (
            "Dear {},\n\n"
            "We are thrilled to inform you of some new and exciting events for our alumni community:\n\n"
            "{}\n\n"
            "We would love for you to join us!\n\n"
            "Best regards,\nAlumni Management Team"
       )

        # Send email to each recipient
        for recipient_name, recipient_email in recipients:
            body = body_template.format(recipient_name, events_info)
            message = "Subject: " + subject + "\n\n" + body  # Explicitly concatenate strings

            server.sendmail(sender_email, recipient_email, message)
            print("Email sent to", recipient_name, "at", recipient_email)

        server.quit()

    except Exception as e:
        print("Failed to send emails:", str(e))

    finally:
        cursor.close()
        db.close()
        Admin2()
```

```python
def search():
    try:
        from utils import get_user_input
        import mysql.connector
        from prettytable import PrettyTable

        db = mysql.connector.connect(host="localhost", user="root", password="1234", database="alumni")
        cursor = db.cursor()

        query = "SELECT student_id, Name, F_Name, M_Name, Passing_Year, Stream, Employment_Status, Company, E_Domain, Email_ID FROM students WHERE 1=1"
        params = []

        print("Enter the search criteria (press Enter to skip any filter):")
        name = get_user_input("Enter the student's name (or press Enter to skip): ", is_name=True)
        f_name = get_user_input("Enter the father's name (or press Enter to skip): ", is_name=True)
        m_name = get_user_input("Enter the mother's name (or press Enter to skip): ", is_name=True)
        passing_year = get_user_input("Enter the passing year (or press Enter to skip): ", is_int=True)
        stream = get_user_input("Enter the stream (or press Enter to skip): ")
        employment_status = get_user_input("Enter the employment status (or press Enter to skip): ")
        company = get_user_input("Enter the company (or press Enter to skip): ")
        domain = get_user_input("Enter the employment domain (or press Enter to skip): ")

        if name:
            query += " AND Name = %s"
            params.append(name)
        if f_name:
            query += " AND F_Name = %s"
            params.append(f_name)
        if m_name:
            query += " AND M_Name = %s"
            params.append(m_name)
        if passing_year:
            query += " AND Passing_Year = %s"
            params.append(passing_year)
        if stream:
            query += " AND Stream = %s"
            params.append(stream)
        if employment_status:
            query += " AND Employment_Status = %s"
            params.append(employment_status)
        if company:
            query += " AND Company = %s"
            params.append(company)
        if domain:
            query += " AND E_Domain = %s"
            params.append(domain)

        cursor.execute(query, tuple(params))
        results = cursor.fetchall()

        table = PrettyTable()
        table.field_names = ["Student ID", "Name", "Father's Name", "Mother's Name", "Passing Year",
                             "Stream", "Employment Status", "Company", "Employment Domain", "Email ID"]

        for result in results:
            table.add_row(result)
        if not results:
            print("No results found.")
        else:
            print("Search Results:")
            print(table)

        return results
    except Exception as e:
        print("An error occurred:", e)
    finally:
        cursor.close()
        db.close()
        Admin2()
```

```
def Alumni():
    from utils import login
    print('''-------------OPTIONS----------------\n
    1. Alumni Registration\n
    2. View Events\n
    3. Event Registration\n
    4. Main Menu\n
    5. Exit''')

    choice = int(input('Enter your choice (1-5):
')) if choice == 1:
        alumni_register()
    elif choice == 2:
        view_events()
    elif choice == 3:
        reg_events()
    elif choice == 4:
        login()
    elif choice == 5:
        exit()
    else:
        print('Invalid input!')
```

```
def alumni_register():
    from utils import is_valid_email, get_input
    import mysql.connector

    db = None
    cursor = None

    try:
        db = mysql.connector.connect(host='localhost', user='root', password='1234', database='alumni')
        cursor = db.cursor()

        while True:
            name = get_input('Enter your name:', is_name=True)

            while True:
                email_input = input("Enter your Email-ID: ")
                if is_valid_email(email_input):
                    email = email_input
                    break
                else:
                    print("Invalid email format. Enter valid Email-ID")

            command = "SELECT * FROM students WHERE Name=%s"
            cursor.execute(command, (name,))
            data = cursor.fetchone()

            if data:
                print("----------WELCOME----------")
                existing_alumni = search_alumni(name=name, email=email)
                if existing_alumni:
                    print("You are already registered as an Alumni!")
                    break
                else:
                    cursor.execute('''INSERT INTO alumni_students (Name, F_Name, Class, DOB, Contact_Number, Email_ID,
                                      Passing_Year, Stream, Current_Country, Current_City,
                                      Employment_Status, E_Domain, Company)
                        VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)''',
                        (name, data[2], data[4], data[5], data[6], email, data[8], data[9], data[10], data[11], data[12], data[13], data[1
4]))
                    db.commit()
                    print('You are successfully registered as an Alumni!')
                    break
            else:
                print("No Data found in students table! Please enter a valid name.")
                break

    except mysql.connector.Error as e:
        print("An error occurred while accessing the database:", e)

    except Exception as e:
        print("An unexpected error occurred:", e)

    finally:
        if cursor:
            cursor.close()
        if db:
            db.close()
        Alumni()
```

```
def view_events():
    from utils import login
    import mysql.connector
    from prettytable import PrettyTable

    db = mysql.connector.connect(host='localhost', user='root', password='1234', database='alumni')
    cursor = db.cursor()

    while True:
        print('------------------------------EVENTS-----------------------------')
        print('''Please Select an Option
        1. Upcoming Events (Active)
        2. Completed Events
        3. Postponed Events
        4. Cancelled Events
        5. Main Menu
        6. Exit''')

        choice = int(input('Enter the option (1-6): '))

        table = PrettyTable()
        table.field_names = ["Event ID", "Event Name", "Event Date", "Venue", "Status"]

        if choice == 1:
            cursor.execute("SELECT Event_ID, Event_Name, Event_Date, Venue, Status FROM event WHERE Status='Active'")
            active_events = cursor.fetchall()
            for event in active_events:
                table.add_row(event)
            print(table)
            break

        elif choice == 2:
            cursor.execute("SELECT Event_ID, Event_Name, Event_Date, Venue, Status FROM event WHERE Status='Complete
d'")
            completed_events = cursor.fetchall()
            for event in completed_events:
                table.add_row(event)
            print(table)
            break

        elif choice == 3:
            cursor.execute("SELECT Event_ID, Event_Name, Event_Date, Venue, Status FROM event WHERE Status='Postpone
d'")
            postponded_events = cursor.fetchall()
            for event in postponded_events:
                table.add_row(event)
            print(table)
            break

        elif choice == 4:
            cursor.execute("SELECT Event_ID, Event_Name, Event_Date, Venue, Status FROM event WHERE Status='Cancelle
d'")
            cancelled_events = cursor.fetchall()
            for event in cancelled_events:
                table.add_row(event)
            print(table)
            break

        elif choice == 5:
            login()
            break

        elif choice == 6:
            cursor.close()
            db.close()
            return

        else:
            print('Invalid choice !!!')

    cursor.close()
    db.close()
    Alumni()
```

```
def reg_events():
    from utils import get_input
    import mysql.connector
    from datetime import datetime
    from prettytable import PrettyTable

    db = mysql.connector.connect(host='localhost', user='root', password='1234', database='alumni')
    cursor = db.cursor()

    cursor.execute("SELECT Event_ID, Event_Name, Event_Date, Available_Seats FROM event WHERE Status='Active' AND Available_Seats > 0")
    events = cursor.fetchall()

    if not events:
        print("No active events with available seats.")
        return

    table = PrettyTable()
    table.field_names = ["Event ID", "Event Name", "Event Date", "Seats Available"]

    for event in events:
        table.add_row(event)

    print('------------------Upcoming Events------------------')
    print(table)

    while True:
        event_id = get_input("Enter the Event ID to register: ", is_int=True)
        cursor.execute("SELECT Event_ID, Available_Seats FROM event WHERE Event_ID=%s AND Status='Active' AND Available_Seats > 0", (event_i
d,))    selected_event = cursor.fetchone()

        if selected_event:
            seats_available = selected_event[1]

            alumni_email = get_input("Enter your Email ID to register for the event: ")
            cursor.execute("SELECT Alumni_ID FROM alumni_students WHERE Email_ID=%s", (alumni_email,))
            alumni_data = cursor.fetchone()

            if alumni_data:
                alumni_id = alumni_data[0]
                registration_date = datetime.now().date()

                cursor.execute('''INSERT INTO event_registration (Event_ID, Alumni_ID, Registration_Date)
                                  VALUES (%s, %s, %s)''', (event_id, alumni_id, registration_date))

                cursor.execute("UPDATE event SET Available_Seats = Available_Seats - 1 WHERE Event_ID = %s", (event_id,))

                db.commit()
                print("You have successfully registered for the event! Seats remaining:", seats_available - 1)
                break
            else:
                print("Alumni not found. Please check your Email ID.")
        else:
            print("Invalid Event ID or no available seats. Please select a valid event.")

    cursor.close()
    db.close()
    Alumni()
```

```
def search_alumni(name=None, email=None, passing_year=None):#Works
    try:
        import mysql.connector
        from prettytable import PrettyTable

        db = mysql.connector.connect(host='localhost', user='root', password='1234', database='alumn
i')     cursor = db.cursor()

        query = "SELECT alumni_id, Name, Email_ID, Passing_Year FROM alumni_students WHERE 1=1"
        parameters = []

        if name:
            query += " AND Name = %s"
            parameters.append(name)
        if email:
            query += " AND Email_ID = %s"
            parameters.append(email)
        if passing_year:
            query += " AND Passing_Year = %s"
            parameters.append(passing_year)

        cursor.execute(query, tuple(parameters))
        results = cursor.fetchall()

        if results:
            table = PrettyTable()
            table.field_names = ["Alumni ID", "Name", "Email ID", "Passing Year"]
            for row in results:
                table.add_row(row)
            print("\nSearch Results:")
            print(table)
            return results
        else:
            print("\nNo matching alumni found.")
            return []

    except mysql.connector.Error as e:
        print("Database error:", e)
        return None
    except Exception as e:
        print("An unexpected error occurred:", e)
        return None
    finally:
        if cursor:
            cursor.close()
        if db:
            db.close()
```

```
import mysql.connector
from datetime import date

def insert_dummy_data():
    db = mysql.connector.connect(host='localhost', user='root', password='1234', database='alumni')
    cursor = db.cursor()

    admin_data = [
        ("admin1", "admin123@"),
        ("admin2", "admin234@"),
        ("admin3", "admin345@"),
        ("admin4", "admin456@"),
        ("admin5", "admin567@")
    ]
    cursor.executemany("INSERT INTO admin_login (username, password) VALUES (%s, %s)", admin_data)

    students_data = [
        ("Rajesh Kumar", "Vijay Kumar", "Sita Devi", "12th", date(2005, 6, 15), "9876543210", "rajeshkumar@gmail.com", 2023, "Science", "India", "Delhi", "Employed", "IT", "TechSolutions"),
        ("Ananya Patel", "Ravi Patel", "Maya Patel", "12th", date(2004, 8, 22), "9876543211", "ananyapatel@yahoo.com", 2022, "Commerce", "India", "Mumbai", "Employed", "Finance", "FinCorp"),
        ("Amit Singh", "Manoj Singh", "Priya Singh", "12th", date(2003, 4, 12), "9876543212", "amitsingh@hotmail.com", 2021, "Arts", "India", "Kolkata", "Unemployed", "Design", "DesignStudio"),
        ("Priya Sharma", "Karan Sharma", "Neelam Sharma", "12th", date(2002, 2, 10), "9876543213", "priyasharma@outlook.com", 2020, "Science", "India", "Bangalore", "Employed", "Engineering", "BuildTech"),
        ("Ravi Mehta", "Vikram Mehta", "Sonia Mehta", "12th", date(2001, 9, 25), "9876543214", "ravimehta@gmail.com", 2019, "Commerce", "India", "Chennai", "Unemployed", "Marketing", "MarketPro")
    ]
    cursor.executemany("INSERT INTO students (Name, F_Name, M_Name, Class, DOB, Contact_Number, Email_ID, Passing_Year, Stream, Current_Country, Current_City, Employment_Status, E_Domain, Company) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)", students_data)

    alumni_data = [
        ("Rajesh Kumar", "Sita Devi", "Vijay Kumar", "12th", date(2005, 6, 15), "9876543210", "rajeshkumar@gmail.com", 2023, "Science", "India", "Delhi", "Employed", "IT", "TechSolutions"),
        ("Ananya Patel", "Maya Patel", "Ravi Patel", "12th", date(2004, 8, 22), "9876543211", "ananyapatel@yahoo.com", 2022, "Commerce", "India", "Mumbai", "Employed", "Finance", "FinCorp"),
        ("Amit Singh", "Priya Singh", "Manoj Singh", "12th", date(2003, 4, 12), "9876543212", "amitsingh@hotmail.com", 2021, "Arts", "India", "Kolkata", "Unemployed", "Design", "DesignStudio"),
        ("Priya Sharma", "Neelam Sharma", "Karan Sharma", "12th", date(2002, 2, 10), "9876543213", "priyasharma@outlook.com", 2020, "Science", "India", "Bangalore", "Employed", "Engineering", "BuildTech"),
        ("Ravi Mehta", "Sonia Mehta", "Vikram Mehta", "12th", date(2001, 9, 25), "9876543214", "ravimehta@gmail.com", 2019, "Commerce", "India", "Chennai", "Unemployed", "Marketing", "MarketPro")
    ]
    cursor.executemany("INSERT INTO alumni_students (Name, M_Name, F_Name, Class, DOB, Contact_Number, Email_ID, Passing_Year, Stream, Current_Country, Current_City, Employment_Status, E_Domain, Company) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)", alumni_dat
a)
    event_data = [
        ("Alumni Meet 2024", date(2024, 5, 10), "Conference", "Grand Hall", 100, 50, "Active"),
        ("Tech Talk 2024", date(2024, 6, 20), "Seminar", "Tech Arena", 200, 150, "Active"),
        ("Networking Event", date(2024, 7, 15), "Networking", "VIP Lounge", 50, 30, "Active"),
        ("Graduation Day", date(2024, 8, 10), "Ceremony", "University Ground", 500, 400, "Active"),
        ("Workshop on AI", date(2024, 9, 5), "Workshop", "Main Auditorium", 120, 100, "Active")
    ]
    cursor.executemany("INSERT INTO event (Event_Name, Event_Date, Type, Venue, Total_Seats, Available_Seats, Status) VALUES (%s, %s, %s, %s, %s, %s, %s)", event_data)

    event_registration_data = [
        (1, 1, date(2024, 4, 25)),
        (2, 2, date(2024, 6, 10)),
        (3, 3, date(2024, 6, 12)),
        (4, 4, date(2024, 8, 1)),
        (5, 5, date(2024, 9, 1))
    ]
    cursor.executemany("INSERT INTO event_registration (event_id, alumni_id, registration_date) VALUES (%s, %s, %s)", event_registration_data)

    db.commit()
    cursor.close()
    db.close()

    print("Dummy data inserted successfully into the tables.")

if __name__ == "__main__":
    insert_dummy_data()
```

Output screen

- **MySQL Tables:**

```
+-------------------+
| Tables_in_alumni  |
+-------------------+
| admin_login       |
| alumni_students   |
| event             |
| event_registration |
| students          |
+-------------------+
```

1. Admin_login Table:

```
+----------+----------+------------+
| admin_id | username | password   |
+----------+----------+------------+
|        1 | admin    | admin123@# |
|        2 | admin1   | admin123#  |
|        3 | admin2   | admin234@# |
|        4 | admin3   | admin345@# |
|        5 | admin4   | admin456@# |
|        6 | admin5   | admin567@# |
+----------+----------+------------+
```

2. Event Table

```
+----------+--------------------------+------------+------------+------------------+-------------+-----------------+-----------+
| Event_ID | Event_Name               | Event_Date | Type       | Venue            | Total_Seats | Available_Seats | Status    |
+----------+--------------------------+------------+------------+------------------+-------------+-----------------+-----------+
|        1 | Alumni Meet 2024         | 2024-05-10 | Conference | Grand Hall       |         100 |              50 | Completed |
|        2 | Tech Talk 2024           | 2024-06-20 | Seminar    | Tech Arena       |         200 |             150 | Completed |
|        3 | Networking Event         | 2024-07-15 | Networking | VIP Lounge       |          50 |              30 | Completed |
|        4 | Graduation Day           | 2024-08-10 | Ceremony   | University Ground |         500 |             400 | Active    |
|        5 | Workshop on AI           | 2024-09-05 | Workshop   | Main Auditorium  |         120 |             100 | Active    |
|        6 | Study Abroad             | 2025-04-03 | Seminar    | Auditorium       |         100 |              98 | Active    |
|        7 | Postgraduate Opportunities | 2025-02-05 | Webinar    | Online (Zoom)    |         100 |              80 | Active    |
+----------+--------------------------+------------+------------+------------------+-------------+-----------------+-----------+
```

3. Event _registration Table

```
+-----------------+----------+-----------+--------------------+
| registration_id | event_id | alumni_id | registration_date  |
+-----------------+----------+-----------+--------------------+
|               1 |        1 |         1 | 2024-04-25         |
|               2 |        2 |         2 | 2024-06-10         |
|               3 |        3 |         3 | 2024-06-12         |
|               4 |        4 |         4 | 2024-08-01         |
|               5 |        5 |         5 | 2024-09-01         |
|               6 |        6 |         6 | 2024-12-09         |
|               7 |        6 |         8 | 2024-12-09         |
+-----------------+----------+-----------+--------------------+
```

4. Alumni_students Table:

```
*************************** 1. row ***************************
        alumni_id: 1
             Name: Rajesh Kumar
          M_Name: Sita Devi
          F_Name: Vijay Kumar
           Class: 12th
             DOB: 2005-06-15
   Contact_Number: 9876543210
         Email_ID: rajeshkumar@gmail.com
     Passing_Year: 2023
           Stream: Science
  Current_Country: India
     Current_City: Delhi
Employment_Status: Employed
         E_Domain: IT
          Company: TechSolutions
*************************** 2. row ***************************
        alumni_id: 2
             Name: Ananya Patel
          M_Name: Maya Patel
          F_Name: Ravi Patel
           Class: 12th
             DOB: 2004-08-22
   Contact_Number: 9876543211
         Email_ID: ananyapatel@yahoo.com
     Passing_Year: 2022
           Stream: Commerce
  Current_Country: India
     Current_City: Mumbai
Employment_Status: Employed
         E_Domain: Finance
          Company: FinCorp
```

```
*************************** 3. row ***************************
        alumni_id: 3
             Name: Amit Singh
          M_Name: Priya Singh
          F_Name: Manoj Singh
           Class: 12th
             DOB: 2003-04-12
   Contact_Number: 9876543212
         Email_ID: amitsingh@hotmail.com
     Passing_Year: 2021
           Stream: Arts
  Current_Country: India
     Current_City: Kolkata
Employment_Status: Unemployed
         E_Domain: Design
          Company: DesignStudio
*************************** 4. row ***************************
        alumni_id: 4
             Name: Priya Sharma
          M_Name: Neelam Sharma
          F_Name: Karan Sharma
           Class: 12th
             DOB: 2002-02-10
   Contact_Number: 9876543213
         Email_ID: priyasharma@outlook.com
     Passing_Year: 2020
           Stream: Science
  Current_Country: India
     Current_City: Bangalore
Employment_Status: Employed
         E_Domain: Engineering
          Company: BuildTech
```

```
*************************** 5. row ***************************
        alumni_id: 5
             Name: Ravi Mehta
           M_Name: Sonia Mehta
           F_Name: Vikram Mehta
            Class: 12th
              DOB: 2001-09-25
   Contact_Number: 9876543214
         Email_ID: ravimehta@gmail.com
     Passing_Year: 2019
           Stream: Commerce
  Current_Country: India
     Current_City: Chennai
Employment_Status: Unemployed
         E_Domain: Marketing
          Company: MarketPro
*************************** 6. row ***************************
        alumni_id: 6
             Name: S Nila
           M_Name: NULL
           F_Name: Geetha
            Class: 12th
              DOB: 2007-11-11
   Contact_Number: 9916416633
         Email_ID: samyukthaschool44@gmail.com
     Passing_Year: 2024
           Stream: Commerce
  Current_Country: India
     Current_City: Bangalore
Employment_Status: Employed
         E_Domain: Accountant
          Company: HDFC BANK
```

```
*************************** 7. row ***************************
        alumni_id: 7
             Name: S Shri Samyuktha
           M_Name: M Shivaraman
           F_Name: V krithika
            Class: 12th
              DOB: 2008-01-03
   Contact_Number: 9620055299
         Email_ID: samyukthaschool44@gmail.com
     Passing_Year: 2025
           Stream: Science
  Current_Country: India
     Current_City: Ranchi
Employment_Status: Employed
         E_Domain: Engineer
          Company: Microsft
*************************** 8. row ***************************
        alumni_id: 8
             Name: Arnav Sinha
           M_Name: Anu Sinha
           F_Name: Santosh Sinha
            Class: 12th
              DOB: 2007-05-02
   Contact_Number: 8951434856
         Email_ID: mynameisarnav02@gmail.com
     Passing_Year: 2025
           Stream: Science
  Current_Country: India
     Current_City: Shimla
Employment_Status: Unemployed
         E_Domain: N/A
          Company: N/A
```

5. Students Table

```
*************************** 1. row ***************************
       student_id: 1
             Name: Rajesh Kumar
           F_Name: Vijay Kumar
           M_Name: Sita Devi
            Class: 12th
              DOB: 2005-06-15
   Contact_Number: 9876543210
         Email_ID: rajeshkumar@gmail.com
     Passing_Year: 2023
           Stream: Science
  Current_Country: India
     Current_City: Delhi
Employment_Status: Employed
         E_Domain: IT
          Company: TechSolutions
*************************** 2. row ***************************
       student_id: 2
             Name: Ananya Patel
           F_Name: Ravi Patel
           M_Name: Maya Patel
            Class: 12th
              DOB: 2004-08-22
   Contact_Number: 9876543211
         Email_ID: ananyapatel@yahoo.com
     Passing_Year: 2022
           Stream: Commerce
  Current_Country: India
     Current_City: Mumbai
Employment_Status: Employed
         E_Domain: Finance
          Company: FinCorp
```

```
*************************** 3. row ***************************
       student_id: 3
             Name: Amit Singh
           F_Name: Manoj Singh
           M_Name: Priya Singh
            Class: 12th
              DOB: 2003-04-12
   Contact_Number: 9876543212
         Email_ID: amitsingh@hotmail.com
     Passing_Year: 2021
           Stream: Arts
  Current_Country: India
     Current_City: Kolkata
Employment_Status: Unemployed
         E_Domain: Design
          Company: DesignStudio
*************************** 4. row ***************************
       student_id: 4
             Name: Priya Sharma
           F_Name: Karan Sharma
           M_Name: Neelam Sharma
            Class: 12th
              DOB: 2002-02-10
   Contact_Number: 9876543213
         Email_ID: priyasharma@outlook.com
     Passing_Year: 2020
           Stream: Science
  Current_Country: India
     Current_City: Bangalore
Employment_Status: Employed
         E_Domain: Engineering
          Company: BuildTech
```

```
*************************** 5. row ***************************
       student_id: 5
             Name: Ravi Mehta
          F_Name: Vikram Mehta
          M_Name: Sonia Mehta
           Class: 12th
             DOB: 2001-09-25
   Contact_Number: 9876543214
         Email_ID: ravimehta@gmail.com
     Passing_Year: 2019
           Stream: Commerce
  Current_Country: India
     Current_City: Chennai
Employment_Status: Unemployed
         E_Domain: Marketing
          Company: MarketPro
*************************** 6. row ***************************
       student_id: 6
             Name: S Nila
          F_Name: Geetha
          M_Name: Suresh
           Class: 12th
             DOB: 2007-11-11
   Contact_Number: 9538527923
         Email_ID: nilas8257@gmail.com
     Passing_Year: 2024
           Stream: Commerce
  Current_Country: India
     Current_City: Bangalore
Employment_Status: Employed
         E_Domain: Accountant
          Company: HDFC BANK
```

```
*************************** 7. row ***************************
       student_id: 7
             Name: S Shri Samyuktha
          F_Name: M Shivaraman
          M_Name: V krithika
           Class: 12th
             DOB: 2008-01-03
   Contact_Number: 9620055299
         Email_ID: samyukthaschool44@gmail.com
     Passing_Year: 2025
           Stream: Science
  Current_Country: India
     Current_City: Ranchi
Employment_Status: Employed
         E_Domain: Engineer
          Company: Microsft
*************************** 8. row ***************************
       student_id: 8
             Name: Arnav Sinha
          F_Name: Anu Sinha
          M_Name: Santosh Sinha
           Class: 12th
             DOB: 2007-05-02
   Contact_Number: 8951434856
         Email_ID: mynameisarnav02@gmail.com
     Passing_Year: 2025
           Stream: Science
  Current_Country: India
     Current_City: Shimla
Employment_Status: Unemployed
         E_Domain: N/A
          Company: N/A
```

PYTHON OUTPUT

- Student Registration

```
Enter A(Admin) or B(Alumni): A
-------------------------LOGIN-------------------------------
Enter your username: Admin

Warning (from warnings module):
  File "C:\Users\muthu\AppData\Local\Programs\Python\Python312\Lib\getpass.py", line 100
    return fallback_getpass(prompt, stream)
GetPassWarning: Can not control echo on the terminal.
Warning: Password input may be echoed.
Enter your password: admin123@#
Login successful!
-------WELCOME----------
Please Select an Option

1. Student Registration

2. Update Student Data

3. Create Event

4. Update Event

5. Send Mail

6. Search Students

7. Main Menu

8. Exit
```

```
Enter a Choice (1-8): 1
Enter the name of the student: Arnav Sinha
Enter the M_Name: Santosh Sinha
Enter the F_Name: Anu Sinha
Enter the class: 12th
Enter the DOB (YYYY-MM-DD): 2007-05-02
Enter the contact number: 8951434856
Enter the email: mynameisarnav02@gmail.com
Enter the passing year: 2025
Enter the stream taken: Science
Enter the Current country of the student: India
Enter the current city of the student: Shimla
Enter Employment status (e.g., Employed, Unemployed, etc.): Unemployed
Enter Domain of work (e.g., HR, Engineer, etc. OR N/A): N/A
Enter the company OR N/A: N/A
Student registered successfully!
+----+-------------+-----------+---------------+-------+------------+----------------+-------------------------+--------------+--------+-----------------+--------------+-------------------+----------+---------+
| ID |     Name    |   M_Name  |     F_Name    | Class |    DOB     | Contact_Number |         Email_ID        | Passing_Year |  Stream | Current_Country | Current_City | Employment_Status | E_Domain | Company |
+----+-------------+-----------+---------------+-------+------------+----------------+-------------------------+--------------+--------+-----------------+--------------+-------------------+----------+---------+
| 8  | Arnav Sinha | Anu Sinha | Santosh Sinha | 12th  | 2007-05-02 |   8951434856   | mynameisarnav02@gmail.com |     2025     | Science |      India      |    Shimla    |     Unemployed    |   N/A    |   N/A   |
+----+-------------+-----------+---------------+-------+------------+----------------+-------------------------+--------------+--------+-----------------+--------------+-------------------+----------+---------+
-------WELCOME----------
Please Select an Option

1. Student Registration

2. Update Student Data

3. Create Event

4. Update Event

5. Send Mail

6. Search Students

7. Main Menu

8. Exit


Enter a Choice (1-8): 2
```

- To Update Student Data

```
Enter a Choice (1-8): 2
Students List:
+------------+-----------------+---------------+---------------+-------+------------+----------------+---------------------------+--------------+---------+-----------------+--------------+-----------------+-------------+---------------+
| student_id |       Name      |    F_Name     |     M_Name    | Class |    DOB     | Contact_Number |          Email_ID         | Passing_Year |  Stream | Current_Country | Current_City | Employment_Status | E_Domain |    Company    |
+------------+-----------------+---------------+---------------+-------+------------+----------------+---------------------------+--------------+---------+-----------------+--------------+-----------------+-------------+---------------+
|     1      |   Rajesh Kumar  | Vijay Kumar   |   Sita Devi   | 12th | 2005-06-15 |   9876543210   |    rajeshkumar@gmail.com  |     2023     | Science |      India      |    Delhi     |     Employed    |      IT     | TechSolutions |
|     2      |   Ananya Patel  | Ravi Patel    |   Maya Patel  | 12th | 2004-08-22 |   9876543211   |    ananyapatel@yahoo.com  |     2022     | Commerce |      India      |    Mumbai    |     Employed    |   Finance   |    FinCorp    |
|     3      |    Amit Singh   | Manoj Singh   |  Priya Singh  | 12th | 2003-04-12 |   9876543212   |    amitsingh@hotmail.com  |     2021     |   Arts  |      India      |   Kolkata    |    Unemployed   |    Design   |  DesignStudio |
|     4      |   Priya Sharma  | Karan Sharma  | Neelam Sharma | 12th | 2002-02-10 |   9876543213   |  priyasharma@outlook.com  |     2020     | Science |      India      |  Bangalore   |     Employed    | Engineering |   BuildTech   |
|     5      |    Ravi Mehta   | Vikram Mehta  |  Sonia Mehta  | 12th | 2001-09-25 |   9876543214   |    ravimehta@gmail.com    |     2019     | Commerce |      India      |   Chennai    |    Unemployed   |  Marketing  |   MarketPro   |
|     6      |      S Nila     |    Geetha     |     Suresh    | 12th | 2007-11-11 |   9916416633   |    nilas8257@gmail.com    |     2024     | Commerce |      India      |  Bangalore   |     Employed    |  Accountant |   HDFC BANK   |
|     7      | S Shri Samyuktha | M Shivaraman |  V krithika   | 12th | 2008-01-03 |   9620055299   | samyukthaschool44@gmail.com |     2025     | Science |      India      |    Ranchi    |     Employed    |   Engineer  |   Microsft    |
|     8      |   Arnav Sinha   | Anu Sinha     | Santosh Sinha | 12th | 2007-05-02 |   8951434856   |  mynameisarnav02@gmail.com |     2025     | Science |      India      |    Shimla    |    Unemployed   |     N/A     |      N/A      |
+------------+-----------------+---------------+---------------+-------+------------+----------------+---------------------------+--------------+---------+-----------------+--------------+-----------------+-------------+---------------+
Select a student by ID to update: 6

Select a student by ID to update: 6
----------------Select option to update-----------------
1: Class
2: Contact Number
3: Email_ID
4: Stream
5: Current Country
6: Current City
7: Employment Status
8: Employment Domain
9: Company

Please enter your Option (1-9): 2
Enter the new contact number: 9538527923
+------------+--------+--------+--------+-------+------------+----------------+-------------------+--------------+---------+-----------------+--------------+-----------------+-------------+-----------+
| student_id |  Name  | F_Name | M_Name | Class |    DOB     | Contact_Number |      Email_ID     | Passing_Year |  Stream | Current_Country | Current_City | Employment_Status | E_Domain |  Company  |
+------------+--------+--------+--------+-------+------------+----------------+-------------------+--------------+---------+-----------------+--------------+-----------------+-------------+-----------+
|     6      | S Nila | Geetha | Suresh |  12th | 2007-11-11 |   9538527923   | nilas8257@gmail.com |     2024     | Commerce |      India      |  Bangalore   |     Employed    |  Accountant | HDFC BANK |
+------------+--------+--------+--------+-------+------------+----------------+-------------------+--------------+---------+-----------------+--------------+-----------------+-------------+-----------+
Student data updated successfully.
-------WELCOME----------
Please Select an Option

1. Student Registration

2. Update Student Data

3. Create Event

4. Update Event

5. Send Mail

6. Search Students

7. Main Menu

8. Exit


Enter a Choice (1-8): 3
```

Creating an Event

```
Enter a Choice (1-8): 3
Enter the Event Name: Postgraduate Opportunities
Enter Date of Event (YYYY-MM-DD): 2025-02-05
Enter the type of event (e.g., Seminar, Workshop, Conference): Webinar
Enter the venue: Online (Zoom)
Enter the total number of seats: 100
Enter the seats available: 80
Enter the status (Active, Cancelled, Postponed, Completed): Active
Event created successfully.
+----------+-------------------------------+------------+---------+-----------------+-------------+-----------------+---------+
| Event_ID |          Event_Name           | Event_Date |   Type  |      Venue      | Total_Seats | Available_Seats | Status  |
+----------+-------------------------------+------------+---------+-----------------+-------------+-----------------+---------+
|     7    | Postgraduate Opportunities | 2025-02-05 | Webinar | Online (Zoom) |     100     |        80       | Active  |
+----------+-------------------------------+------------+---------+-----------------+-------------+-----------------+---------+
-------WELCOME----------
Please Select an Option

1. Student Registration

2. Update Student Data

3. Create Event

4. Update Event

5. Send Mail

6. Search Students

7. Main Menu

8. Exit


Enter a Choice (1-8): 4
```

- To Update Event

```
Enter a Choice (1-8): 4
Existing Events:
+----------+-----------------------------+------------+------------+--------------------+-------------+-----------------+-----------+
| Event_ID |         Event_Name          | Event_Date |    Type    |       Venue        | Total_Seats | Available_Seats |   Status  |
+----------+-----------------------------+------------+------------+--------------------+-------------+-----------------+-----------+
|    1     |      Alumni Meet 2024       | 2024-05-10 | Conference |     Grand Hall     |     100     |        50       | Completed |
|    2     |       Tech Talk 2024        | 2024-06-20 |  Seminar   |     Tech Arena     |     200     |       150       | Completed |
|    3     |      Networking Event       | 2024-07-15 | Networking |     VIP Lounge     |      50     |        30       |   Active  |
|    4     |       Graduation Day        | 2024-08-10 |  Ceremony  | University Ground  |     500     |       400       |   Active  |
|    5     |       Workshop on AI        | 2024-09-05 |  Workshop  |  Main Auditorium   |     120     |       100       |   Active  |
|    6     |        Study Abroad         | 2025-04-03 |  Seminar   |     Auditorium     |     100     |        99       |   Active  |
|    7     | Postgraduate Opportunities  | 2025-02-05 |  Webinar   |   Online (Zoom)    |     100     |        80       |   Active  |
+----------+-----------------------------+------------+------------+--------------------+-------------+-----------------+-----------+

Enter the Event ID to update: 3
Enter the new status (Active, Cancelled, Postponed, Completed) or press Enter to keep current: Completed
Enter new Date of Event (YYYY-MM-DD) or press Enter to keep current:
Enter the new venue or press Enter to keep current:
Event updated successfully.

Updated Event Details:
+----------+------------------+------------+------------+------------+-------------+-----------------+-----------+
| Event ID |    Event Name    | Event Date |    Type    |   Venue    | Total Seats | Available Seats |   Status  |
+----------+------------------+------------+------------+------------+-------------+-----------------+-----------+
|    3     | Networking Event | 2024-07-15 | Networking | VIP Lounge |      50     |        30       | Completed |
+----------+------------------+------------+------------+------------+-------------+-----------------+-----------+
-------WELCOME----------
Please Select an Option

1. Student Registration

2. Update Student Data

3. Create Event

4. Update Event

5. Send Mail

6. Search Students

7. Main Menu

8. Exit



Enter a Choice (1-8): 5
```

- To Send Mail

```
Enter a Choice (1-8): 5

Active Events:
+----------+-------------------------------+------------+--------------------+
| Event ID |          Event Name           | Event Date |       Venue        |
+----------+-------------------------------+------------+--------------------+
|    1     |       Alumni Meet 2024        | 2024-05-10 |     Grand Hall     |
|    2     |        Tech Talk 2024         | 2024-06-20 |     Tech Arena     |
|    3     |       Networking Event        | 2024-07-15 |     VIP Lounge     |
|    4     |        Graduation Day         | 2024-08-10 | University Ground  |
|    5     |        Workshop on AI         | 2024-09-05 |  Main Auditorium   |
|    6     |         Study Abroad          | 2025-04-03 |     Auditorium     |
|    7     | Postgraduate Opportunities    | 2025-02-05 |   Online (Zoom)    |
+----------+-------------------------------+------------+--------------------+
Do you want to send details of these active events to the alumni? (Y/N): y

Search Results:
+-----------+--------------+-------------------------------+---------------+
| Alumni ID |     Name     |            Email ID           | Passing Year  |
+-----------+--------------+-------------------------------+---------------+
|     1     | Rajesh Kumar |    rajeshkumar@gmail.com      |     2023      |
|     2     | Ananya Patel |    ananyapatel@yahoo.com      |     2022      |
|     3     |  Amit Singh  |    amitsingh@hotmail.com      |     2021      |
|     4     | Priya Sharma |   priyasharma@outlook.com     |     2020      |
|     5     |  Ravi Mehta  |     ravimehta@gmail.com       |     2019      |
|     6     |    S Nila    | samyukthaschool44@gmail.com   |     2024      |
+-----------+--------------+-------------------------------+---------------+

Selected Alumni:
+----+--------------+-------------------------------+
| ID |     Name     |             Email             |
+----+--------------+-------------------------------+
| 1  | Rajesh Kumar |    rajeshkumar@gmail.com      |
| 2  | Ananya Patel |    ananyapatel@yahoo.com      |
| 3  |  Amit Singh  |    amitsingh@hotmail.com      |
| 4  | Priya Sharma |   priyasharma@outlook.com     |
| 5  |  Ravi Mehta  |     ravimehta@gmail.com       |
| 6  |    S Nila    | samyukthaschool44@gmail.com   |
+----+--------------+-------------------------------+
Do you want to send emails to the selected alumni? (Y/N): y
Email sent to Rajesh Kumar at rajeshkumar@gmail.com
Email sent to Ananya Patel at ananyapatel@yahoo.com
Email sent to Amit Singh at amitsingh@hotmail.com
Email sent to Priya Sharma at priyasharma@outlook.com
Email sent to Ravi Mehta at ravimehta@gmail.com
Email sent to S Nila at samyukthaschool44@gmail.com
-------WELCOME----------
```

```
-------WELCOME----------
Please Select an Option

1. Student Registration

2. Update Student Data

3. Create Event

4. Update Event

5. Send Mail

6. Search Students

7. Main Menu

8. Exit



Enter a Choice (1-8): 6
```

- To Search Students

```
Enter a Choice (1-8): 6
Enter the search criteria (press Enter to skip any filter):
Enter the student's name (or press Enter to skip):
Enter the father's name (or press Enter to skip):
Enter the mother's name (or press Enter to skip):
Enter the passing year (or press Enter to skip):
Enter the stream (or press Enter to skip): science
Enter the employment status (or press Enter to skip):
Enter the company (or press Enter to skip):
Enter the employment domain (or press Enter to skip):
Search Results:
+------------+-----------------+---------------+----------------+--------------+---------+-------------------+-------------+-----------------+-------------------------+
| Student ID |       Name      | Father's Name | Mother's Name  | Passing Year |  Stream | Employment Status |   Company   | Employment Domain |         Email ID        |
+------------+-----------------+---------------+----------------+--------------+---------+-------------------+-------------+-----------------+-------------------------+
|     1      |   Rajesh Kumar  |  Vijay Kumar  |   Sita Devi    |     2023     | Science |      Employed     | TechSolutions |       IT        |   rajeshkumar@gmail.com   |
|     4      |   Priya Sharma  |  Karan Sharma | Neelam Sharma  |     2020     | Science |      Employed     |  BuildTech  |   Engineering   |  priyasharma@outlook.com  |
|     7      | S Shri Samyuktha |  M Shivaraman |   V krithika   |     2025     | Science |      Employed     |   Microsft  |     Engineer    | samyukthaschool44@gmail.com |
|     8      |   Arnav Sinha   |   Anu Sinha   | Santosh Sinha  |     2025     | Science |     Unemployed    |     N/A     |       N/A       |  mynameisarnav02@gmail.com  |
+------------+-----------------+---------------+----------------+--------------+---------+-------------------+-------------+-----------------+-------------------------+
-------WELCOME----------
Please Select an Option

1. Student Registration

2. Update Student Data

3. Create Event

4. Update Event

5. Send Mail

6. Search Students

7. Main Menu

8. Exit



Enter a Choice (1-8): 7
```

- Registering as Alumni

```
Enter A(Admin) or B(Alumni): b
-------------OPTIONS-----------------

    1. Alumni Registration

    2. View Events

    3. Event Registration

    4. Main Menu

    5. Exit
Enter your choice (1-5): 1
Enter your name:S Shri Samyuktha
Enter your Email-ID: samyukthaschool44@gmail.com
----------WELCOME----------

Search Results:
+------------+-------------------+-----------------------------+----------------+
| Alumni ID |        Name        |           Email ID          | Passing Year |
+------------+-------------------+-----------------------------+----------------+
|     9      | S Shri Samyuktha | samyukthaschool44@gmail.com |      2025      |
+------------+-------------------+-----------------------------+----------------+
You are already registered as an Alumni!
-------------OPTIONS-----------------

    1. Alumni Registration

    2. View Events

    3. Event Registration

    4. Main Menu

    5. Exit
Enter your choice (1-5): 2
```

- To View Events

```
------------------------------EVENTS-----------------------------
Please Select an Option
        1. Upcoming Events (Active)
        2. Completed Events
        3. Postponed Events
        4. Cancelled Events
        5. Main Menu
        6. Exit
Enter the option (1-6): 1
+----------+----------------------------+------------+--------------------+--------+
| Event ID |         Event Name         | Event Date |       Venue        | Status |
+----------+----------------------------+------------+--------------------+--------+
|    4     |       Graduation Day       | 2024-08-10 | University Ground  | Active |
|    5     |       Workshop on AI       | 2024-09-05 |  Main Auditorium   | Active |
|    6     |        Study Abroad        | 2025-04-03 |     Auditorium     | Active |
|    7     | Postgraduate Opportunities | 2025-02-05 |   Online (Zoom)    | Active |
+----------+----------------------------+------------+--------------------+--------+
-------------OPTIONS-----------------

    1. Alumni Registration

    2. View Events

    3. Event Registration

    4. Main Menu

    5. Exit
Enter your choice (1-5): 3
```

- To Register for Event

```
    5. Exit
Enter your choice (1-5): 3
-------------------Upcoming Events-------------------
+----------+----------------------------+------------+-----------------+
| Event ID |         Event Name         | Event Date | Seats Available |
+----------+----------------------------+------------+-----------------+
|    4     |       Graduation Day       | 2024-08-10 |       400       |
|    5     |       Workshop on AI       | 2024-09-05 |       100       |
|    6     |        Study Abroad        | 2025-04-03 |        99       |
|    7     | Postgraduate Opportunities | 2025-02-05 |        80       |
+----------+----------------------------+------------+-----------------+
Enter the Event ID to register: 6
Enter your Email ID to register for the event: mynameisarnav02@gmail.com
You have successfully registered for the event! Seats remaining: 98
-------------OPTIONS-----------------

    1. Alumni Registration

    2. View Events

    3. Event Registration

    4. Main Menu

    5. Exit
Enter your choice (1-5): 5
>>>
```

Conclusion

The Alumni Management Program is a comprehensive solution designed to streamline and strengthen the connection between an institution and its alumni. With features catering to both administrators and alumni, the program efficiently handles critical operations such as student and alumni registration, event management, and communication.

Key Achievements:

- **Enhanced Engagement:** By enabling alumni to register, view, and participate in events, the program fosters stronger alumni relations and encourages active participation.
- **Streamlined Administration:** Administrators can easily manage student and alumni records, organise events, and communicate effectively through an integrated email system.
- **Data Integrity and Security:** The program validates all user inputs and incorporates secure database connections, ensuring the reliability and confidentiality of data.
- **User-Friendly Interface:** Simple navigation and clear options make the program accessible to users of varying technical expertise.

Scalability and Future Prospects:

- The program can be scaled further to include additional features such as fundraising campaigns, alumni job portals, or mentorship programs.
- Integration with social media platforms could enhance connectivity and communication within the alumni network.

- Implementing analytics tools can provide insights into alumni engagement trends and event participation.

In conclusion, the Alumni Management Program bridges the gap between the institution and its alumni, creating a robust network that benefits both parties while laying the groundwork for future enhancements and expansions

Bibliography

1. Primary source:
   - Sumita Arora class 12 computer science
2. Secondary source:
   - https://www.geeksforgeeks.org/
   - https://www.w3schools.com/